

AZ-Delivery

Bienvenue !

Merci d'avoir acheté notre *AZ-Delivery ESP-32 Dev Kit C V4*. Dans les pages suivantes, nous allons vous présenter comment utiliser et configurer cet appareil pratique.

Amusez-vous bien !

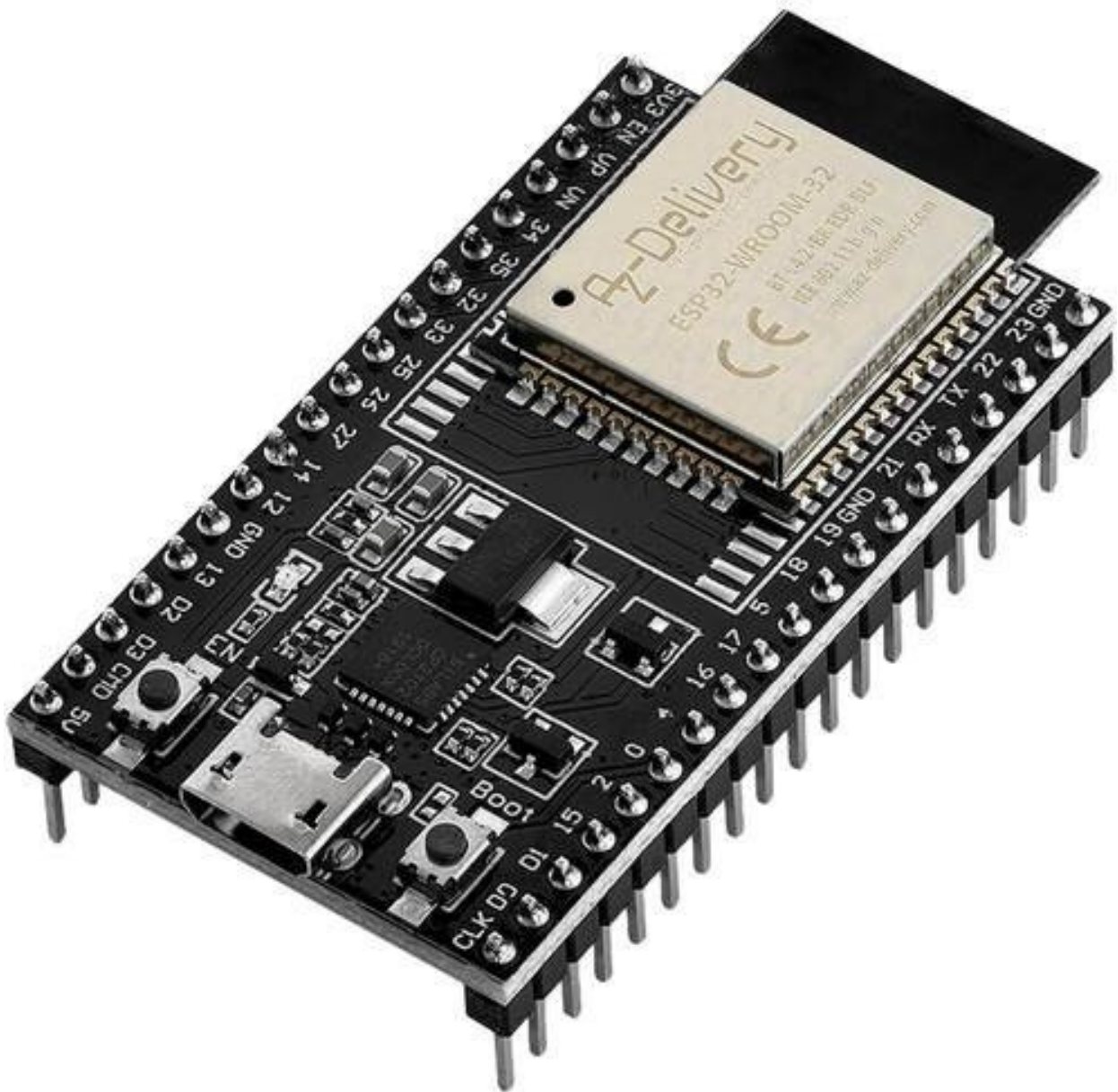


Table des matières

Introduction.....	3
Spécifications	4
ESP32 Dev Kit C V4	5
Brochage	7
Description des broches d'entrée et de sortie	8
Broches du capteur tactile capacitif.....	9
Broches du convertisseur analogique-numérique	10
Broches du convertisseur numérique-analogique	10
Horloge temps réel Broches GPIO.....	11
Broches PWM (modulation de largeur d'impulsion)	12
Les broches de l'interface I2C.....	12
Broches d'interface SPI.....	13
Goupilles de cerclage	13
Épingles HAUTES à la botte	14
Validation (EN)	14
Communication USB-série	15
Communication WiFi	16
Communication Bluetooth.....	17
Autres caractéristiques.....	19
Comment configurer l'IDE Arduino.....	20
Configuration supplémentaire	24
ESP32 Dev Kit C V4 exemple de câblage.....	28
Exemples de croquis	29



Introduction

L'ESP32 Dev Kit C V4 est une carte de développement créée autour de la puce ESP32 Wroom 32D, contenant un régulateur de tension et un circuit de programmation USB pour la puce ESP32, ainsi que de nombreuses autres fonctionnalités.

Pour le développement d'applications, il est possible de choisir entre Arduino IDE et ESP-IDF (plateforme native). La plupart des utilisateurs choisissent l'Arduino IDE en raison de sa simplicité et de sa compatibilité. La communauté des utilisateurs d'Arduino est très active et soutient des plateformes telles que l'ESP32.

L'ESP32 Dev Kit C V4 est livré avec un micrologiciel préinstallé qui permet de travailler avec le langage interprété, en envoyant des commandes via le port série (puce *CP2102*). La carte ESP32 Dev Kit C V4 est l'une des plateformes les plus utilisées pour les projets de l'Internet des objets (IoT).

La carte ESP32 Dev Kit C V4 est spécialement conçue pour fonctionner sur une planche à pain. Elle possède un régulateur de tension qui lui permet d'être alimentée directement par le port USB. Les broches d'entrée/sortie fonctionnent à 3,3V. La puce *CP2102* est responsable de la communication USB-série.



Spécifications

Tension d'alimentation (USB)	5V DC
Tension d'entrée/sortie	3,3V DC
Courant de fonctionnement requis	min. 500mA
SoC	ESP32-WROOM-32D
UNITÉ CENTRALE	Xtensa® à double cœur 32 bits LX6
Plage de fréquence de l'horloge	80MHz / 240MHz
RAM	512kB
Mémoire flash externe	4MB
Broches E/S	34
Canaux ADC	18
Résolution ADC	12 bits
Canaux DAC	2
Résolution DAC	8 bits
Interfaces de communication	SPI, I2C, I2S, CAN, UART
Protocoles Wi-Fi	802.11 b/g/n (802.11n jusqu'à 150 Mbps)
Fréquence Wi-Fi	2,4 GHz - 2,5 GHz
Bluetooth	V4.2 - BLE et Bluetooth classique
Antenne sans fil	PCB



ESP32 Dev Kit C V4

La série de puces Wi-Fi ESP32 WROOM 32D est produite par Espressif Systems. L'ESP32 WROOM-32D est un module Wi-Fi abordable adapté aux projets de bricolage dans le domaine de l'Internet des objets (IoT). Ce module est doté de nombreux GPIO et prend en charge une variété de protocoles tels que SPI, I2C, I2S, UART, etc. La meilleure partie est qu'il est livré avec un réseau sans fil inclus, ce qui le rend différent des autres microcontrôleurs comme l'Arduino. Cela signifie qu'il peut facilement contrôler et surveiller des appareils à distance via Wi-Fi et Bluetooth® à un prix abordable.

L'ESP32 WROOM-32D est un système sur puce (SoC) intégrant un microcontrôleur Tensilica 32 bits, des interfaces périphériques numériques standard, des commutateurs d'antenne, un balun RF, un amplificateur de puissance, un amplificateur de réception à faible bruit, des filtres et des modules de gestion de l'énergie dans un petit boîtier. Il offre une communication sans fil 2.4GHz Wi-Fi (802.11 b/g/n, supportant des vitesses jusqu'à 150MB/s), BLE et Bluetooth® classique, 34 broches E/S, interfaces I2C et I2S, ADC (conversion analogique-numérique), DAC (conversion numérique-analogique), interface SPI, UART sur des broches dédiées, et PWM (modulation de largeur d'impulsions).

Le cœur du processeur, appelé LX6 par Espressif, est basé sur le contrôleur de processeur Xtensa® dual-core 32-bit LX6 et fonctionne à une fréquence comprise entre 80 et 240MHz. Il dispose d'une ROM de

Az-Delivery

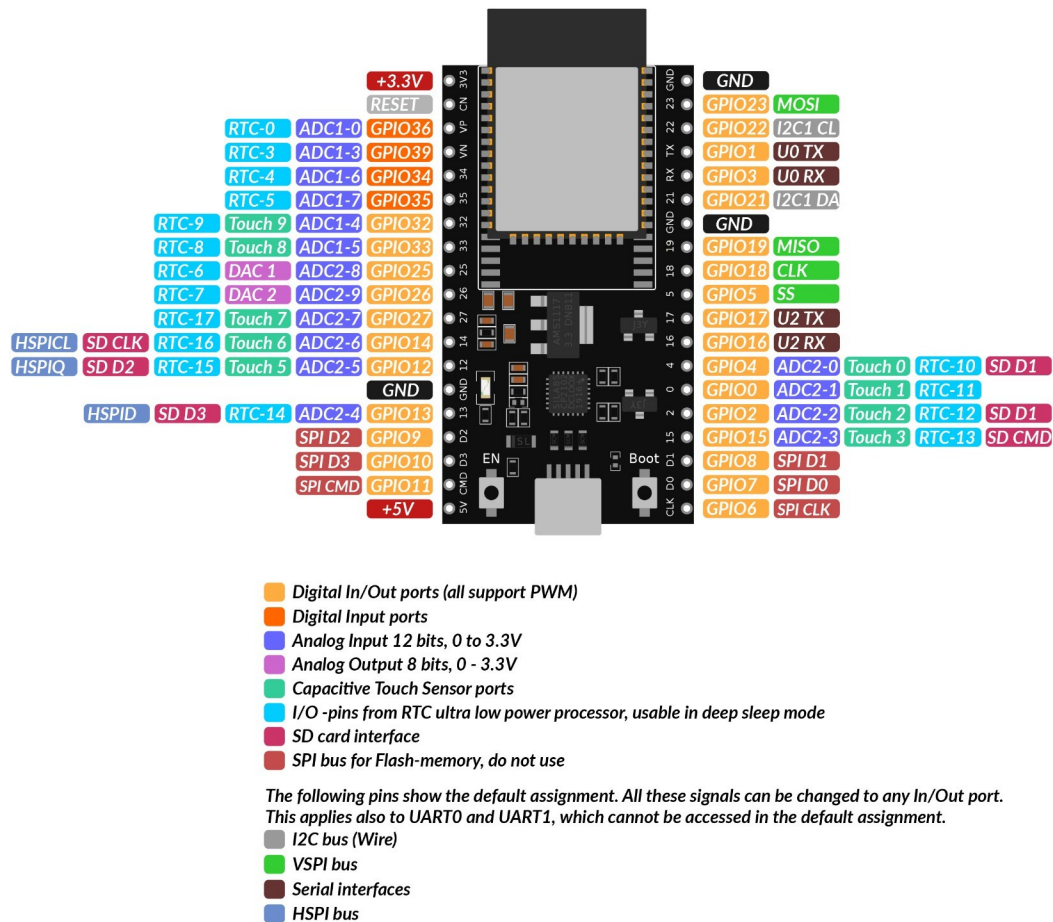
démarrage de 448kB, de 520kB de SRAM on-chip et de 4MB de mémoire flash externe accessible via l'interface SPI.



Les fournisseurs ont donc créé de nombreux modules compacts basés sur la puce ESP32 WROOM-32D. Certains de ces modules ont des identifiants spécifiques.

Brochage

L'ESP32 Dev Kit C V4 possède 38 broches. Le brochage est indiqué sur l'image suivante :



Pour une description détaillée du brochage et des capacités d'E/S, veuillez vous référer à la fiche technique qui se trouve sur le [lien](#) suivant.

REMARQUE : Le courant maximal absolu consommé par un GPIO est de 40 mA, conformément à la section "Recommended Operating Conditions" (conditions de fonctionnement recommandées) de la fiche technique de l'ESP32.



Description des broches d'entrée et de sortie

Tout comme une carte Arduino normale, l'ESP32 Dev Kit C V4 possède des broches d'entrée/sortie numériques (broches GPIO - General Purpose Input/Output pins). Ces entrées/sorties numériques fonctionnent à 3,3 V.

La tension de 5V ne doit pas être connectée aux broches de la puce ESP32 !

Les broches ne sont pas tolérantes à 5V, l'application de plus de 3,3V sur une broche détruira la puce.

Les broches GPIO 34 à 39 sont des GPI - broches d'entrée uniquement. Ces broches n'ont pas de résistances internes d'excursion vers le haut ou vers le bas. Elles ne peuvent pas être utilisées en tant que sorties, il faut donc les utiliser uniquement en tant qu'entrées : GPIO 34, GPIO 35, GPIO 36, GPIO 39

La puce ESP-WROOM-32 intègre un flash SPI. Les broches GPIO6 à GPIO 11 sont exposées dans certaines cartes de développement ESP32. Ces broches sont connectées au flash SPI intégré sur la puce et ne sont pas recommandées pour d'autres usages.

GPIO 6 (SCK/CLK), GPIO 7 (SDO/SD0), GPIO 8 (SDI/SD1), GPIO 9 (SHD/SD2), GPIO 10 (SWP/SD3), GPIO 11 (CSC/CMD).



Broches du capteur tactile capacitif

L'ESP32 possède 10 capteurs tactiles capacitifs internes. Les broches tactiles capacitives peuvent également être utilisées pour réveiller l'ESP32 d'un sommeil profond. Ces capteurs tactiles internes sont connectés aux GPIO suivants : T0 (GPIO 4), T1 (GPIO 0), T2 (GPIO 2), T3 (GPIO 15), T4 (GPIO 13), T5 (GPIO 12), T6 (GPIO 14), T7 (GPIO 27), T8 (GPIO 33), T9 (GPIO 32).

Broches du convertisseur analogique-numérique

L'ESP32 dispose de 18 canaux d'entrée ADC (convertisseur analogique-numérique) de 12 bits (alors que l'ESP8266 ne dispose que de 1 ADC de 10 bits). Voici les GPIO qui peuvent être utilisés comme ADC et les canaux correspondants :

ADC1_CH0 (GPIO 36), ADC1_CH1 (GPIO 37), ADC1_CH2 (GPIO 38),
ADC1_CH3 (GPIO 39), ADC1_CH4 (GPIO 32), ADC1_CH5 (GPIO 33),
ADC1_CH6 (GPIO 34), ADC1_CH7 (GPIO 35), ADC2_CH0 (GPIO 4),
ADC2_CH1 (GPIO 0), ADC2_CH2 (GPIO 2), ADC2_CH3 (GPIO 15),
ADC2_CH4 (GPIO 13), ADC2_CH5 (GPIO 12), ADC2_CH6 (GPIO 14),
ADC2_CH7 (GPIO 27), ADC2_CH8 (GPIO 25), ADC2_CH9 (GPIO 26).

Broches du convertisseur numérique-analogique

L'ESP32 dispose de 2 canaux DAC (convertisseur numérique-analogique) de 8 bits pour convertir les signaux numériques en signaux de sortie analogiques. Il s'agit des canaux DAC :

DAC1 (GPIO25), DAC2 (GPIO26).



Horloge temps réel Broches GPIO

L'ESP32 prend en charge les GPIO de l'horloge en temps réel (RTC). Les GPIO acheminés vers le sous-système RTC basse consommation peuvent être utilisés lorsque l'ESP32 est en sommeil profond. Ces GPIO RTC peuvent être utilisées pour réveiller l'ESP32 lorsque le coprocesseur Ultra Low Power (ULP) est en cours d'exécution. Les GPIO suivants peuvent être utilisés comme source de réveil externe : RTC_GPIO0 (GPIO36), RTC_GPIO3 (GPIO39), RTC_GPIO4 (GPIO34), RTC_GPIO5 (GPIO35), RTC_GPIO6 (GPIO25), RTC_GPIO7 (GPIO26), RTC_GPIO8 (GPIO33), RTC_GPIO9 (GPIO32), RTC_GPIO10 (GPIO4), RTC_GPIO11 (GPIO0), RTC_GPIO12 (GPIO2), RTC_GPIO13 (GPIO15), RTC_GPIO14 (GPIO13), RTC_GPIO15 (GPIO12), RTC_GPIO16 (GPIO14), RTC_GPIO17 (GPIO27).

Broches PWM (modulation de largeur d'impulsion)

Le contrôleur PWM (Pulse width modulation) de l'ESP32 LED possède 16 canaux indépendants qui peuvent être configurés pour générer des signaux PWM avec différentes propriétés. Toutes les broches pouvant servir de sorties peuvent être utilisées comme broches PWM (les GPIO 34 à 39 ne peuvent pas générer de PWM). Pour configurer un signal PWM, vous devez définir les paramètres suivants dans le code : Fréquence du signal, rapport cyclique, canal PWM, GPIO où vous souhaitez émettre le signal.

Les broches de l'interface I2C

L'ESP32 possède deux canaux I2C et n'importe quelle broche peut être définie comme SDA ou SCL. Lorsque l'ESP32 est utilisé avec l'IDE Arduino, les broches I2C par défaut sont :
GPIO 21 (SDA), GPIO 22 (SCL).

Broches d'interface SPI

Par défaut, le mappage des broches SPI est le suivant :

SPI	MOSI	MISO	CLK	CS
VSPI	GPIO 23	GPIO 19	GPIO 18	GPIO 5
HSPI	GPIO 13	GPIO 12	GPIO 14	GPIO 15

Cheilles de cerclage

Les broches suivantes sont utilisées pour mettre l'ESP32 en mode chargeur de démarrage ou en mode clignotant : GPIO 0, GPIO 2, GPIO 4, GPIO 5 (doit être HIGH pendant le démarrage), GPIO 12 (doit être LOW pendant le démarrage), GPIO 15 (doit être HIGH pendant le démarrage).

La plupart des cartes de développement mettent les broches dans le bon état pour le flashage ou le mode de démarrage. Si certains périphériques sont connectés aux broches de strapping et que l'IDE est incapable de télécharger le code ou de flasher l'ESP32, cela peut être dû au fait que ces périphériques empêchent l'ESP32 d'entrer dans le bon mode. Après la réinitialisation, le flashage ou le démarrage, ces broches fonctionnent comme prévu. Un guide de documentation sur la sélection du mode de démarrage est disponible sur le [lien](#) suivant. Des explications plus approfondies ne font pas partie du champ d'application de cet eBook, il convient donc de se référer à la fiche technique.



Pins HIGH at Boot

Certains GPIO passent à l'état HAUT ou émettent des signaux PWM au démarrage ou à la réinitialisation. Cela signifie que si des sorties sont connectées à ces GPIO, des résultats inattendus peuvent être obtenus lorsque l'ESP32 se réinitialise ou démarre.

GPIO 1, GPIO 3, GPIO 5, GPIO 6 à GPIO 11 (connecté à la mémoire flash SPI intégrée de l'ESP32 - utilisation non recommandée), GPIO 14, GPIO 15.

Validation (EN)

Enable (EN) est la broche d'activation du régulateur 3,3V. Elle est tirée vers le haut et doit être connectée à la masse pour désactiver le régulateur 3,3V.

Cela signifie que cette broche peut être connectée à un bouton poussoir pour redémarrer votre ESP32, par exemple.



Communication USB-série

L'ESP32 Dev Kit C V4 dispose d'un port de connexion microUSB. Il est composé d'une puce CP21202 fabriquée par Silicon Laboratories qui permet une communication série USB vers UART. La puce possède la fonction de port COM virtuel (VCP) qui apparaît comme un port COM dans les applications PC. L'interface UART CP2102 met en œuvre tous les signaux RS-232, y compris les signaux de contrôle et d'échange de mains, de sorte qu'il n'est pas nécessaire de modifier le micrologiciel du système existant. Pour pouvoir utiliser l'ESP32, le pilote doit être installé.



Communication WiFi

L'ESP32 Dev Kit C V4 dispose d'une interface de communication Wi-Fi intégrée et peut fonctionner dans trois modes différents : station Wi-Fi, point d'accès Wi-Fi et les deux en même temps. Il prend en charge les fonctions suivantes :

- Débits de données 802.11b et 802.11g
- 802.11n MCS0-7 dans une largeur de bande de 20 MHz et 40 MHz
- 802.11n MCS32
- 802.11n 0,4μS intervalle de garde
- Débit de données jusqu'à 150 Mbps
- Réception STBC 2x1
- Puissance d'émission jusqu'à 20 dBm
- Puissance d'émission réglable
- Diversité et sélection des antennes (matériel géré par logiciel)

Communication Bluetooth

L'ESP32 Dev Kit C V4 est doté d'une radio Bluetooth intégrée et prend en charge les fonctions suivantes :

- Puissances de sortie d'émission de classe 1, de classe 2 et de classe 3 et plage de contrôle dynamique de plus de 30 dB
- Modulation $\pi/4$ DQPSK et 8 DPSK
- Haute performance en matière de sensibilité du récepteur NZIF avec une plage dynamique de plus de 98 dB
- Fonctionnement en classe 1 sans sonorisation externe
- La SRAM interne permet un transfert de données à grande vitesse, un mélange de voix et de données, et un fonctionnement en piconet complet.
- Logique pour la correction d'erreurs vers l'avant, le contrôle des erreurs d'en-tête, la corrélation des codes d'accès, le CRC, la démodulation, la génération de flux de bits de cryptage, le blanchiment et la mise en forme des impulsions de transmission.
- ACL, SCO, eSCO et AFH
- CODEC audio numérique A-law, μ -law et CVSD dans l'interface PCM
- CODEC audio SBC
- Gestion de l'énergie pour les applications à faible consommation
- SMP avec AES 128 bits

La radio Bluetooth prend également en charge les protocoles d'interface de communication suivants :

- Interface UART HCI, jusqu'à 4 Mbps
- Interface SDIO / SPI HCI
- Interface I2C
- Interface audio PCM / I2S.

Autres caractéristiques

La puce ESP32-WROOM 32D intègre un capteur à effet Hall qui détecte les variations du champ magnétique dans son environnement.

Le capteur à effet Hall est basé sur une résistance à porteur N. Lorsque la puce se trouve dans le champ magnétique, le capteur à effet Hall développe une petite tension sur la résistance, qui peut être mesurée directement par le convertisseur analogique-numérique (CAN), ou amplifiée par le préamplificateur analogique à très faible bruit, puis mesurée par le CAN.

Le capteur de température génère une tension qui varie en fonction de la température. La tension est convertie en interne en un code numérique via un convertisseur analogique-numérique. Le capteur de température a une plage de -40°C à 125°C . Étant donné que le décalage du capteur de température varie d'une puce à l'autre en raison de la variation du processus, ainsi que de la chaleur générée par le circuit Wi-Fi lui-même (qui affecte les mesures), le capteur de température interne ne convient qu'aux applications qui détectent des changements de température plutôt que des températures absolues, ainsi qu'à des fins d'étalonnage. Toutefois, si l'utilisateur étalonne le capteur de température et utilise l'appareil dans une application à alimentation minimale, les résultats peuvent être suffisamment précis.

Comment configurer l'IDE Arduino

Si l'Arduino IDE n'est pas installé, suivez le [lien](#) et téléchargez le fichier d'installation pour le système d'exploitation de votre choix. La version de l'Arduino IDE utilisée pour cet eBook est **1.8.13**.

Download the Arduino IDE



ARDUINO 1.8.13

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software. This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions.

Windows Installer, for Windows 7 and up
Windows ZIP file for non admin install

Windows app Requires Win 8.1 or 10 [Get](#)

Mac OS X 10.10 or newer

Linux 32 bits
Linux 64 bits
Linux ARM 32 bits
Linux ARM 64 bits

[Release Notes](#)
[Source Code](#)
[Checksums \(sha512\)](#)

Pour les utilisateurs de *Windows*, double-cliquez sur le fichier `.exe` téléchargé et suivez les instructions de la fenêtre d'installation.

Az-Delivery

Pour les utilisateurs de *Linux*, téléchargez un fichier avec l'extension *.tar.xz*, qui doit être extrait. Une fois le fichier extrait, allez dans le répertoire extrait et ouvrez le terminal dans ce répertoire. Deux scripts *.sh* doivent être exécutés, le premier appelé *arduino-linux-setup.sh* et le second appelé *install.sh*.

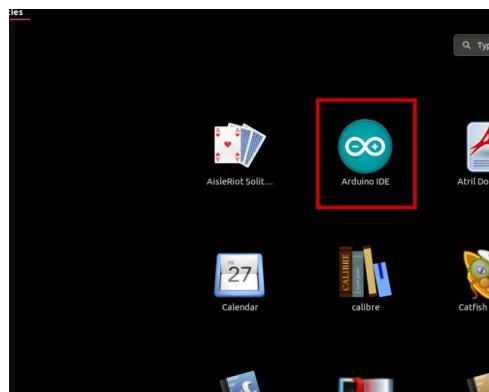
Pour exécuter le premier script dans le terminal, ouvrez le terminal dans le répertoire extrait et exécutez la commande suivante :

```
sh arduino-linux-setup.sh nom_utilisateur
```

nom_utilisateur - est le nom d'un superutilisateur dans le système d'exploitation Linux. Le mot de passe du superutilisateur doit être saisi lors du lancement de la commande. Attendez quelques minutes que le script ait terminé.

Le second script, appelé *install.sh*, doit être utilisé après l'installation du premier script. Exécutez la commande suivante dans le terminal (répertoire extrait) : **sh install.sh**

Après l'installation de ces scripts, allez dans *All Apps*, où l'*IDE Arduino* est installé.



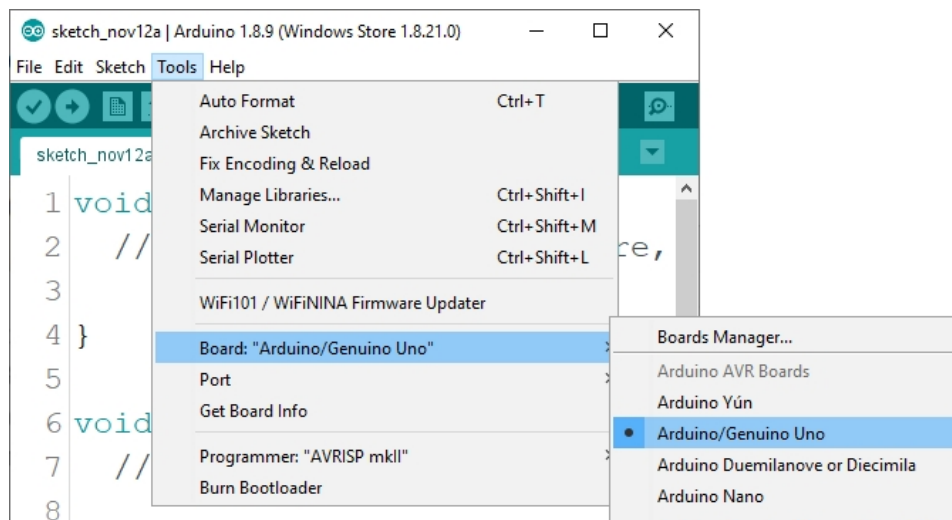
Az-Delivery

Presque tous les systèmes d'exploitation sont livrés avec un éditeur de texte préinstallé (par exemple, *Windows* est livré avec *Notepad*, *Linux Ubuntu* est livré avec *Gedit*, *Linux Raspbian* est livré avec *Leafpad*, etc.) Tous ces éditeurs de texte conviennent parfaitement à l'objectif du livre électronique.

La prochaine chose à faire est de vérifier si votre PC peut détecter une carte Arduino. Ouvrez l'IDE Arduino fraîchement installé, et allez dans :

Outils > Tableau > {votre nom de tableau ici}

{votre nom de carte ici} devrait être l'*Arduino/Genuino Uno*, comme on peut le voir sur l'image suivante :

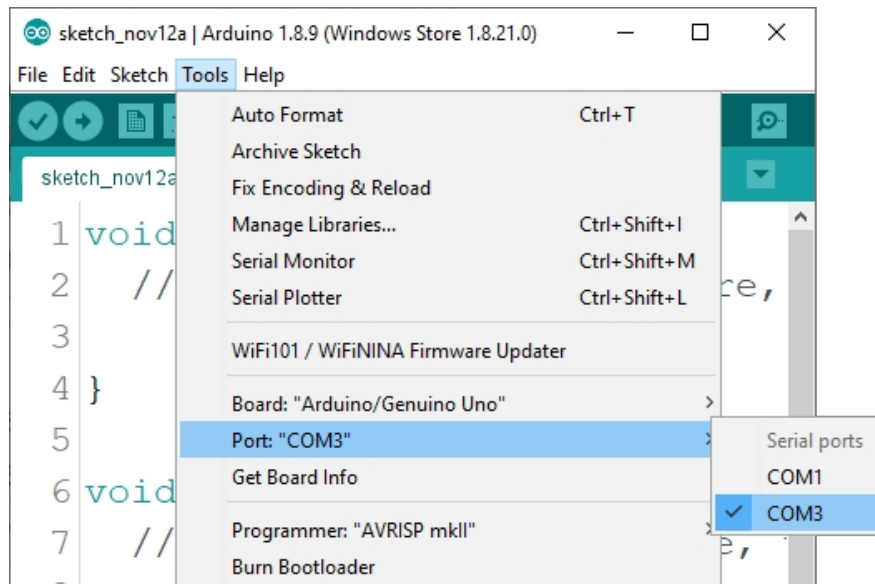


Le port auquel la carte Arduino est connectée doit être sélectionné. Aller à :

Outils > Port > {le nom du port est ici}

et lorsque la carte Arduino est connectée au port USB, le nom du port est visible dans le menu déroulant de l'image précédente.

Si l'IDE Arduino est utilisé sous Windows, les noms des ports sont les suivants :



Pour les utilisateurs de *Linux*, le nom du port est par exemple `/dev/ttyUSBx`, où `x` représente un nombre entier entre 0 et 9.



Configuration supplémentaire

Pour utiliser l'ESP32 Dev Kit C V4 avec Arduino IDE, suivez quelques étapes simples. Avant de configurer l'Arduino IDE, le pilote pour la communication USB-série doit être installé. Si le pilote n'est pas installé automatiquement, il existe une page de support qui contient les pilotes pour Windows/Mac ou Linux et qui peut être choisie en fonction de celui qui est utilisé. Les pilotes peuvent être téléchargés à partir du [lien](#) suivant.

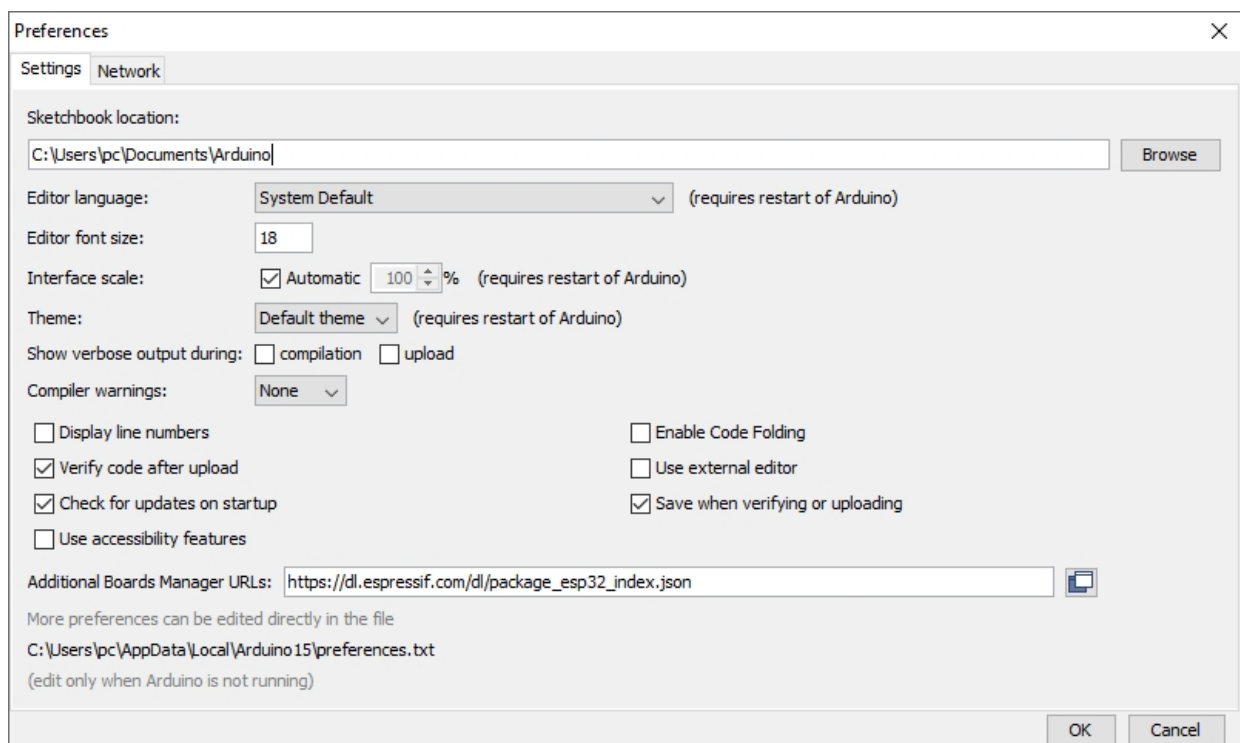
Az-Delivery

Ensuite, pour installer le support pour la plateforme ESP32, ouvrez l'IDE Arduino et allez dans :

Fichier > Préférences, et trouvez le champ URL supplémentaires.

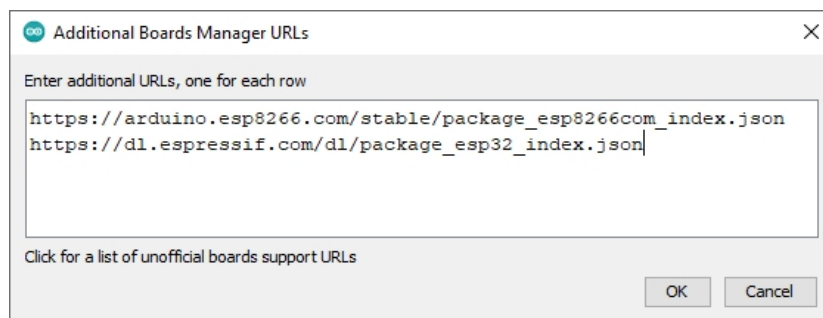
Copiez ensuite l'URL suivante :

https://dl.espressif.com/dl/package_esp32_index.json



Az-Delivery

Collez ce lien dans le champ URL supplémentaires. Si un ou plusieurs liens se trouvent dans ce champ, il suffit d'ajouter une virgule après le dernier lien, de coller le nouveau lien après la virgule et de cliquer sur le bouton **OK**.



Ouvrez à nouveau l'IDE Arduino et allez sur :

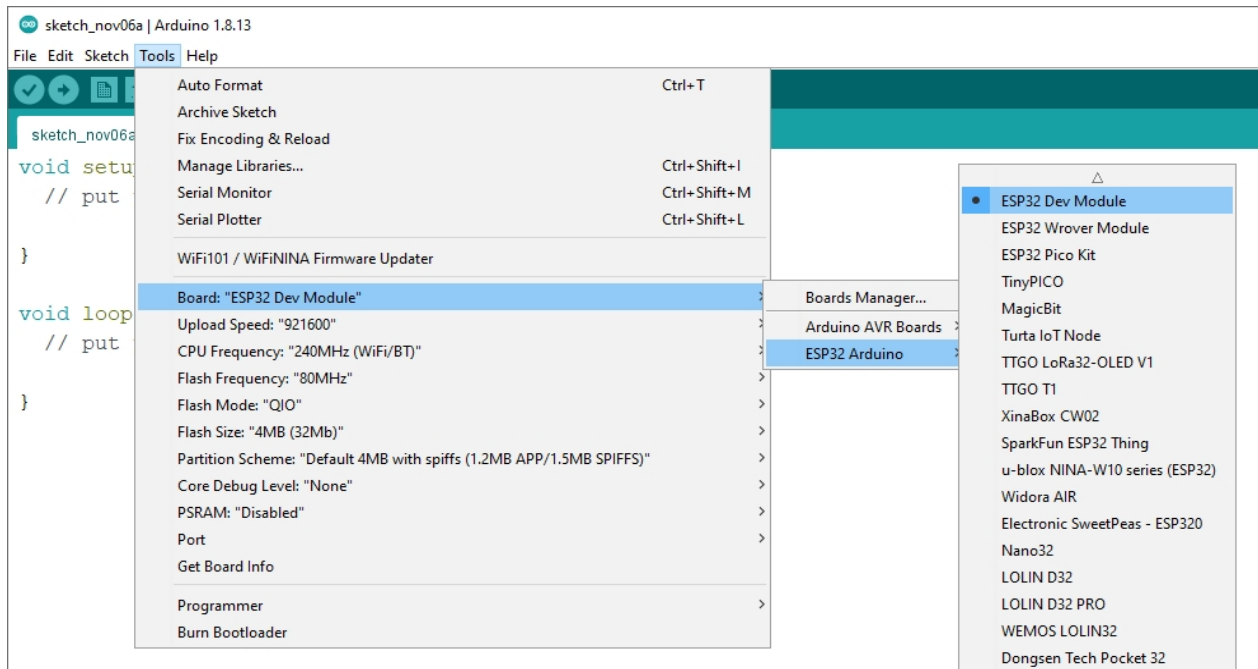
Outils > Tableau > Gestionnaire de tableaux

Lorsque la nouvelle fenêtre s'ouvre, tapez **esp32** dans la boîte de recherche et installez la carte **esp32** fabriquée par *Espressif Systems*, comme indiqué sur l'image suivante :

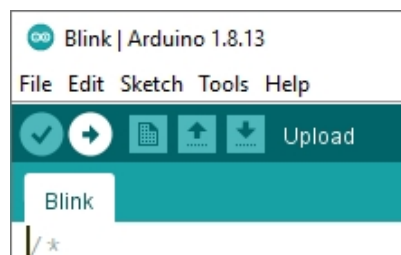


Pour sélectionner la carte ESP32, rendez-vous sur le site :

Outils > Carte > ESP32 Arduino > ESP32 Dev Module

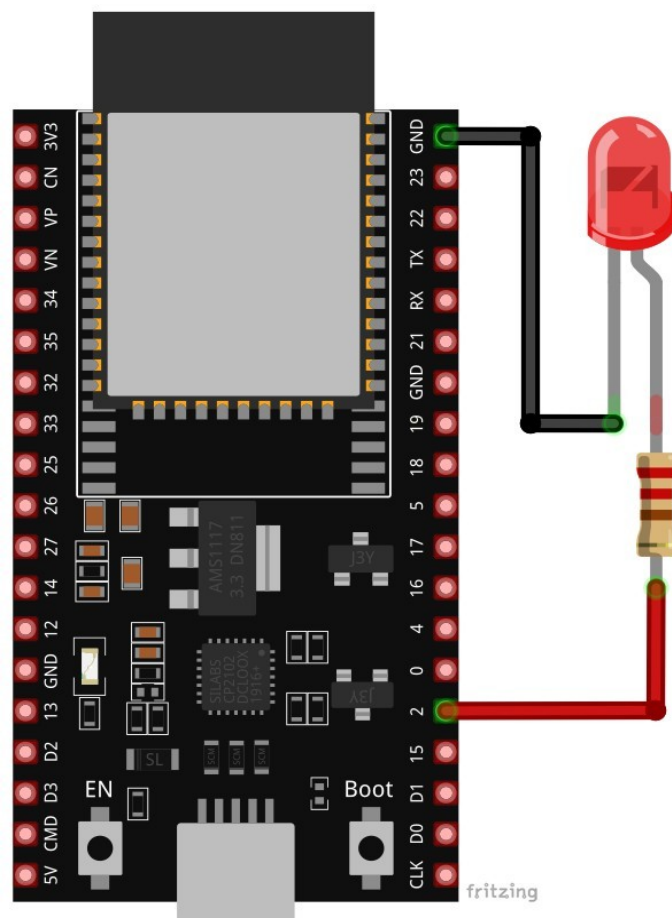


Pour télécharger le code de l'esquisse sur la carte ESP32, sélectionnez d'abord le port sur lequel vous avez connecté la carte. Allez dans : *Outils > Port > {nom du port}*



Exemple de câblage de l'ESP32 Dev Kit C V4

Connectez l'ESP32 Dev Kit C V4 avec une LED et une résistance comme indiqué dans le diagramme de connexion suivant :



ESP32 Dev Kit C V4 pin	Broche LED	Couleur du fil
GPIO2 (pin2)	Anode (+) à travers la résistance	Fil rouge
GND	Cathode (-)	Fil noir

Exemples de croquis

LED clignotante

```
int ledPin = 2 ;

void setup() {
  pinMode(ledPin, OUTPUT) ;
}

void loop() {
  digitalWrite(ledPin, HIGH) ;
  delay(1000) ;
  digitalWrite(ledPin, LOW) ;
  delay(1000) ;
}
```



PWM - Modulation de largeur d'impulsion

```
#define LEDC_CHANNEL_0 0
#define LEDC_TIMER_13_BIT 13
#define LEDC_BASE_FREQ 5000
#define LED_PIN 2

int brightness = 0 ;
int fadeAmount = 5 ;

void ledcAnalogWrite(uint8_t channel, uint32_t value, uint32_t valueMax = 255)
{
    uint32_t duty = (8191 / valueMax) * min(value, valueMax) ;
    ledcWrite(channel, duty) ;
}

void setup() {
    ledcSetup(LEDC_CHANNEL_0, LEDC_BASE_FREQ, LEDC_TIMER_13_BIT) ;
    ledcAttachPin(LED_PIN, LEDC_CHANNEL_0) ;
}

void loop() {
    ledcAnalogWrite(LEDC_CHANNEL_0, brightness) ;
    brightness = brightness + fadeAmount ;
    if (brightness <= 0 || brightness >= 255) {
        fadeAmount = -fadeAmount ;
    }
    delay(30) ;
}
```



Il est maintenant temps d'apprendre et de créer vos propres projets. Vous pouvez le faire avec l'aide de nombreux scripts d'exemple et d'autres tutoriels, que vous pouvez trouver sur Internet.

Si vous recherchez des produits de haute qualité pour Arduino et Raspberry Pi, AZ-Delivery Vertriebs GmbH est l'entreprise qu'il vous faut. Vous disposerez de nombreux exemples d'application, de guides d'installation complets, de livres électroniques, de bibliothèques et de l'assistance de nos experts techniques.

<https://az-delivery.de>

Amusez-vous !

Impressum

<https://az-delivery.de/pages/about-us>